

Les objets prédéfinis dans Thymeleaf

1. Présentation

Thymeleaf met à disposition des **objets prédéfinis** que l'on peut utiliser partout dans un template, à l'intérieur des expressions `${...}`. On distingue deux familles :

- les **objets de base** (*basic objects*), qui donnent accès au contexte d'exécution (locale, contexte de traitement...);
- les **objets utilitaires** (*utility objects*), qui fournissent des méthodes pour manipuler des dates, des nombres, des chaînes, des collections, etc.

Ces objets sont référencés par un nom commençant par le symbole #, par exemple `#{#strings.isEmpty(name)}`. Dans une application Spring, les expressions sont évaluées par SpringEL ; le préfixe # y désigne une variable du contexte d'évaluation, ce qui est le cas de ces objets.

Changement majeur de Thymeleaf 3.1 (Spring Boot 3 et suivants). Les objets `#request`, `#response`, `#session` et `#servletContext`, qui donnaient accès à l'API Servlet, ont été **supprimés** des expressions, pour des raisons de sécurité (éviter l'accès à des propriétés potentiellement dangereuses, comme les paramètres de requête) et parce qu'ils étaient liés à l'API `javax.*`. Un template qui les utilise lève une exception au rendu : « *The 'request', 'session', 'servletContext' and 'response' expression utility objects are no longer available...* ». La section 3 montre comment les remplacer. De nombreux tutoriels anciens les utilisent encore : il faut s'en méfier.

2. Les objets de base

Objet	Classe / interface	Description
<code>#ctx</code>	<code>org.thymeleaf.context.IContext</code>	Le contexte de traitement du template
<code>#locale</code>	<code>java.util.Locale</code>	La locale (langue, pays) utilisée pour le rendu

Les objets suivants **existaient avant la version 3.1** et ne sont plus disponibles : `#request` (`HttpServletRequest`), `#response` (`HttpServletResponse`), `#session` (`HttpSession`) et `#servletContext` (`ServletContext`). Dans Spring Boot 3, ces classes appartiennent au paquet `jakarta.servlet.*`.

`#ctx`

`#ctx` est l'objet de contexte. Il permet par exemple de lister les noms des variables disponibles ou d'obtenir la locale :

```
<p th:text="${#ctx.locale}"></p>
<p th:text="${#ctx.variableNames}"></p>
```

L'accès aux objets web via `#ctx` (par exemple `${#ctx.request}`) relève de la même restriction que `#request` : il est à proscrire. Les anciens objets `#vars` et `#root`, similaires à `#ctx`, sont obsolètes : n'utilisez plus que `#ctx`, et seulement lorsque c'est nécessaire.

#locale

`#locale` fournit des informations sur l'environnement régional de la requête : pays, langue, nom affichable, etc. Ces informations servent par exemple au format des nombres et des dates ou à l'internationalisation.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title>Objets prédéfinis</title>
</head>
<body>
  <h1>Objet prédéfini #locale</h1>

  <h3><code>#locale.country</code></h3>
  <span th:text="${#locale.country}"></span>

  <h3><code>#locale.language</code></h3>
  <span th:text="${#locale.language}"></span>

  <h3><code>#locale.displayName</code></h3>
  <span th:text="${#locale.displayName}"></span>
</body>
</html>
```

Avec un navigateur configuré en français (France), on obtient FR, fr et « français (France) ».

3. Remplacer `#request`, `#session`, `#response` et `#servletContext`

Le principe est simple : ce n'est plus le template qui va chercher les données dans l'API Servlet, c'est le **contrôleur** (ou un composant dédié) qui prépare ce dont la vue a besoin et le place dans le modèle. On sépare ainsi plus nettement la logique de la présentation.

Ancien usage	Remplacement recommandé
<code>\${#request.contextPath}</code>	Utiliser les expressions d'URL <code>@{/chemin}</code> : le chemin de contexte est ajouté automatiquement
<code>\${#request.requestURI}</code>	L'ajouter au modèle (contrôleur ou <code>@ControllerAdvice</code>)
<code>\${#request.getParameter('q')}</code>	<code>\${param.q}</code>

Ancien usage	Remplacement recommandé
<code>\${#session.getAttribute('k')}</code>	<code>\${session.k}</code>
<code>\${#servletContext.getAttribute('k')}</code>	<code>\${application.k}</code>
<code>\${#request.headerNames}</code>	Récupérer les en-têtes dans le contrôleur (<code>@RequestHeader</code>)

Exemples

Session : le contrôleur place une valeur dans la session, le template la lit avec la variable `session`.

```
@GetMapping("/predefined-object-session")
public String objectSession(HttpSession session) {
    session.setAttribute("mygreeting", "Hello Everyone!");
    return "predefined-object-session";
}

<h1>Attribut de session</h1>
<span th:text="${session.mygreeting}"></span>
```

Paramètre de requête : pour l'URL `/search?q=thymeleaf`,

```
<p>Recherche : <span th:text="${param.q}"></span></p>
```

`${param.q}` est un tableau de chaînes (un paramètre peut être répété) ; pour une seule valeur, utilisez `${param.q[0]}`, ou testez sa présence avec `${param.containsKey('q')}`.

URI courante : un `@ControllerAdvice` peut ajouter la même information à tous les modèles, ce qui évite de modifier chaque contrôleur (par exemple pour mettre en évidence l'élément de menu actif).

```
@ControllerAdvice
public class GlobalModelAttributes {

    @ModelAttribute("currentUri")
    public String currentUri(HttpServletRequest request) {
        return request.getRequestURI();
    }
}

<a th:href="@{/persons}"
    th:classappend="${#strings.startsWith(currentUri, '/persons')} ?
    'active'">
    Personnes
</a>
```

En-têtes de la requête : le contrôleur les passe explicitement au modèle.

```
@GetMapping("/predefined-object-headers")
public String headers(@RequestHeader Map<String, String> headers, Model
```

```

model) {
    model.addAttribute("headers", headers);
    return "predefined-object-headers";
}

<ul>
    <li th:each="h : ${headers}" th:text="${h.key} + ' : ' + ${h.value}">En-
tête</li>
</ul>

```

Sécurité. N'exposez à la vue que le nécessaire. Afficher sans précaution des entêtes, des cookies ou des paramètres peut révéler des informations sensibles. C'est précisément l'une des raisons de la suppression de ces objets.

4. Les objets utilitaires

Objet	Classe	Description
#execInfo	org.thymeleaf.expression.ExecutionInfo	Informations sur le template en cours de traitement
#messages	org.thymeleaf.expression.Messages	Accès aux messages internationalisés
#uris	org.thymeleaf.expression.Uris	Échappement des parties d'URL/URI
#conversions	org.thymeleaf.expression.Conversions	Service de conversion configuré (le cas échéant)
#temporals	org.thymeleaf.expression.Temporals	Formatage et extraction pour l'API java.time (LocalDate, LocalDateTime, ...)
#dates	org.thymeleaf.expression.Dates	Formatage et extraction pour java.util.Date
#calendars	org.thymeleaf.expression.Calendars	Analogue à #dates, pour java.util.Calendar
#numbers	org.thymeleaf.expression.Numbers	Formatage des nombres
#strings	org.thymeleaf.expression.Strings	Méthodes sur les chaînes (contains, startsWith...)
#objects	org.thymeleaf.expression.Objects	Méthodes sur les objets en général
#bools	org.thymeleaf.expression.Bools	Évaluation de booléens
#arrays	org.thymeleaf.expression.Arrays	Méthodes sur les tableaux
#lists	org.thymeleaf.expression.Lists	Méthodes sur les listes
#sets	org.thymeleaf.expression.Sets	Méthodes sur les ensembles

Objet	Classe	Description
#maps	org.thymeleaf.expression.Maps	Méthodes sur les dictionnaires
#aggregates	org.thymeleaf.expression.Aggregates	Sommes, moyennes... sur une collection ou un tableau
#ids	org.thymeleaf.expression.Ids	Gestion d'attributs id pouvant être répétés (dans une itération, par exemple)

Chaque objet propose bien plus de méthodes que celles présentées ci-dessous : consultez la documentation officielle (section *Appendix B: Expression Utility Objects*) pour la liste complète.

#execInfo

Informations sur le template en cours de traitement.

```
@GetMapping("/predefined-u-object-execInfo")
public String execInfo() {
    return "predefined-u-object-execInfo";
}
```

<h3><code>#execInfo.templateMode</code></h3>

<h3><code>#execInfo.templateName</code></h3>

<h3><code>#execInfo.now</code> (<code>java.util.Calendar</code></h3>

Remarque : depuis Thymeleaf 3.1, la variable `{#execInfo}` n'existe plus ; seule la forme `{#execInfo}` est disponible.

#uris

Échappement de parties d'URL, pour construire des liens sûrs. On utilise `escapeQueryParam` pour la valeur d'un paramètre de requête et `escapePathSegment` pour un segment de chemin.

<h4><code>#uris.escapeQueryParam('Tom & Jerry')</code></h4>

<h4><code>#uris.unescapeQueryParam('Tom%20%26%20Jerry')</code></h4>

La première expression produit Tom%20%26%20Jerry, la seconde Tom & Jerry. Pour construire une URL complète, préférez néanmoins la syntaxe `@{/search(q=${term})}`, qui encode automatiquement les paramètres.

#temporals (API java.time)

Depuis Thymeleaf 3.1, l'objet `#temporals` est **toujours disponible** (le module complémentaire `thymeleaf-extras-java8time` a été intégré au cœur). C'est l'objet à privilégier pour les types `LocalDate`, `LocalDateTime`, `Instant`, etc., qui ont remplacé `Date` et `Calendar` dans le Java moderne.

```
<th:block th:with="now = ${#temporals.createNow()}">
  <h4><code>#temporals.format(now, 'dd/MM/yyyy HH:mm:ss')</code></h4>
  <span th:text="${#temporals.format(now, 'dd/MM/yyyy HH:mm:ss')}"></span>

  <h4><code>#temporals.year(now)</code></h4>
  <span th:text="${#temporals.year(now)}"></span>

  <h4><code>#temporals.month(now)</code></h4>
  <span th:text="${#temporals.month(now)}"></span>
</th:block>
```

Pour un objet de votre modèle, par exemple `order.createdAt` de type `LocalDateTime` :

```
th:text="${#temporals.format(order.createdAt, 'dd/MM/yyyy')}"
```

#dates et #calendars (anciennes API)

Ces objets ont les mêmes fonctions pour `java.util.Date` et `java.util.Calendar`. On les rencontre dans du code existant, mais pour un nouveau développement on choisit `java.time` et `#temporals`.

```
<th:block th:with="now = ${#dates.createNow()}">
  <span th:text="${#dates.format(now, 'yyyy-MM-dd HH:mm:ss')}"></span>
  <span th:text="${#dates.year(now)}"></span>
  <span th:text="${#dates.month(now)}"></span>
</th:block>

<th:block th:with="now = ${#calendars.createNow()}">
  <span th:text="${#calendars.format(now, 'yyyy-MM-dd HH:mm:ss')}"></span>
  <span th:text="${#calendars.year(now)}"></span>
  <span th:text="${#calendars.month(now)}"></span>
</th:block>
```

Attention : chaque objet doit être utilisé avec le type correspondant (`#dates` pour `Date`, `#calendars` pour `Calendar`).

#numbers

Formatage des nombres. Le résultat dépend de la locale, sauf si l'on précise les séparateurs.

```

<th:block th:with="num = 12345.987654">
  <h4><code>num</code></h4>
  <span th:text="{num}"></span>

  <!-- 3 chiffres minimum pour la partie entière -->
  <h4><code>#numbers.formatInteger(num, 3)</code></h4>
  <span th:text="{#numbers.formatInteger(num, 3)}"></span>

  <!-- séparateur de milliers : point -->
  <h4><code>#numbers.formatInteger(num, 3, 'POINT')</code></h4>
  <span th:text="{#numbers.formatInteger(num, 3, 'POINT')}"></span>

  <!-- milliers : point, 2 décimales, séparateur décimal : virgule -->
  <h4><code>#numbers.formatDecimal(num, 3, 'POINT', 2, 'COMMA')</code></h4>
  <span th:text="{#numbers.formatDecimal(num, 3, 'POINT', 2, 'COMMA')}"></span>
</th:block>

```

Les paramètres sont : le nombre, le nombre minimal de chiffres de la partie entière, le séparateur de milliers (POINT, COMMA, WHITESPACE, NONE, DEFAULT), puis, pour formatDecimal, le nombre de décimales et le séparateur décimal. La dernière expression produit 12.345,99.

Autres méthodes utiles : #numbers.formatCurrency(price) (selon la locale), #numbers.formatPercent(rate, 1, 1), #numbers.sequence(1, 5) (liste de 1 à 5, pratique avec th:each).

#strings

Les méthodes les plus utilisées :

Expression	Rôle
{#strings.isEmpty(s)}	Vrai si s est null ou vide
{#strings.contains(s, 'abc')}	Contient une sous-chaîne
{#strings.startsWith(s, 'ab')} / endsWith	Début / fin de chaîne
{#strings.toUpperCase(s)} / toLowerCase	Changement de casse
{#strings.capitalize(s)}	Première lettre en majuscule
{#strings.length(s)}	Longueur
{#strings.substring(s, 0, 3)}	Extraction
{#strings.replace(s, 'a', 'b')}	Remplacement
{#strings.abbreviate(s, 20)}	Troncature avec des points de suspension
{#strings.listSplit(s, ',')}	Découpage en liste
{#strings.listJoin(list, ',')}	Assemblage d'une liste

```
<p th:text="${#strings.toUpperCase('thymeleaf')}"></p>    <!-- THYMELEAF -->
<p th:text="${#strings.capitalize('thymeleaf')}"></p>    <!-- ThymeLeaf -->
<p th:if="${#strings.isEmpty(user.nickname)}">Pas de pseudo</p>
```

Chaque méthode existe aussi en version array..., list... et set... pour traiter une collection de chaînes d'un coup.

#objects

Méthodes pour les objets en général. Exemple : `nullSafe` et `arrayNullSafe` remplacent les valeurs `null` par une valeur par défaut.

```
@GetMapping("/predefined-u-object-objects")
public String objects(Model model) {
    // Un tableau contenant des valeurs null
    String[] colors = {"red", "blue", null, "green", null, "red"};
    model.addAttribute("colors", colors);
    return "predefined-u-object-objects";
}

<h4><code>#objects.arrayNullSafe(colors, 'white')</code></h4>
<ul>
    <li th:each="color : ${#objects.arrayNullSafe(colors, 'white')}"
        th:text="${color}"></li>
</ul>
```

Les valeurs `null` sont remplacées par `white` : on obtient `red, blue, white, green, white, red`. Pour un seul objet, `${#objects.nullSafe(user.nickname, 'Anonyme')}` est une alternative à l'opérateur Elvis.

#bools

Évaluation de booléens, avec les règles de véracité de Thymeleaf (`null`, `false`, `0`, `"false"`, `"off"`, `"no"` sont faux). Les méthodes `arrayIsTrue` et `arrayIsFalse` évaluent **chaque élément** d'un tableau et renvoient un tableau de booléens.

```
@GetMapping("/predefined-u-object-bools")
public String bools(Model model) {
    String[] colors = {"red", null, "blue"};
    model.addAttribute("colors", colors);
    return "predefined-u-object-bools";
}

<h4>Tableau : colors</h4>
<ul><li th:each="color : ${colors}" th:text="${color}"></li></ul>

<h4><code>#bools.arrayIsTrue(colors)</code></h4>
<ul><li th:each="b : ${#bools.arrayIsTrue(colors)}" th:text="${b}"></li></ul>

<h4><code>#bools.arrayIsFalse(colors)</code></h4>
```

```
<ul><li th:each="b : ${#bools.arrayIsFalse(colors)}"
th:text="${b}"></li></ul>
```

Le premier résultat est true, false, true ; le second false, true, false. Les méthodes arrayAnd et arrayOr (et leurs équivalents list... et set...) combinent les éléments par un « et » ou un « ou » logique.

#arrays

```
@GetMapping("/predefined-u-object-arrays")
public String arrays(Model model) {
    String[] colors = {"red", null, "blue"};
    model.addAttribute("colors", colors);
    return "predefined-u-object-arrays";
}
```

```
<p>Vide ? <span th:text="${#arrays.isEmpty(colors)}"></span></p>
<p>Longueur : <span th:text="${#arrays.length(colors)}"></span></p>
<p>Contient « red » ? <span th:text="${#arrays.contains(colors,
'red')}"></span></p>
```

Résultats : false, 3 et true.

#lists

```
@GetMapping("/predefined-u-object-lists")
public String lists(Model model) {
    model.addAttribute("colors", List.of("red", "blue", "green"));
    return "predefined-u-object-lists";
}
```

```
<h4><code>#lists.sort(colors)</code></h4>
<ul><li th:each="c : ${#lists.sort(colors)}" th:text="${c}"></li></ul>

<p>Vide ? <span th:text="${#lists.isEmpty(colors)}"></span></p>
<p>Taille : <span th:text="${#lists.size(colors)}"></span></p>
<p>Contient « red » ? <span th:text="${#lists.contains(colors,
'red')}"></span></p>
```

#lists.sort renvoie une **nouvelle liste triée** (par ordre alphabétique : blue, green, red) sans modifier la liste d'origine. Les résultats suivants sont false, 3 et true.

#sets

```
@GetMapping("/predefined-u-object-sets")
public String sets(Model model) {
    // LinkedHashSet : ordre d'insertion conservé (un HashSet n'a aucun ordre garanti)
    Set<String> colors = new LinkedHashSet<>(List.of("red", "blue", "green"));
    model.addAttribute("colors", colors);
    return "predefined-u-object-sets";
}
```

```
<p>Vide ? <span th:text="${#sets.isEmpty(colors)}"></span></p>
<p>Taille : <span th:text="${#sets.size(colors)}"></span></p>
<p>Contient « red » ? <span th:text="${#sets.contains(colors,
'red')}"></span></p>
```

#maps

```
@GetMapping("/predefined-u-object-maps")
```

```
public String maps(Model model) {
    Map<String, String> contacts = new LinkedHashMap<>();
    contacts.put("111 222", "Tom");
    contacts.put("111 333", "Jerry");
    contacts.put("111 444", "Donald");
    model.addAttribute("contacts", contacts);
    return "predefined-u-object-maps";
}
```

```
<p>Vide ? <span th:text="${#maps.isEmpty(contacts)}"></span></p>
<p>Taille : <span th:text="${#maps.size(contacts)}"></span></p>
<p>Contient la clé « 111 222 » ? <span th:text="${#maps.containsKey(contacts,
'111 222')}"></span></p>
```

#aggregates

Somme, moyenne... sur un tableau ou une collection de nombres.

```
@GetMapping("/predefined-u-object-aggregates")
```

```
public String aggregates(Model model) {
    double[] salaries = {100, 200, 500};
    model.addAttribute("salaries", salaries);
    return "predefined-u-object-aggregates";
}
```

```
<h4><code>#aggregates.sum(salaries)</code></h4>
<span th:text="${#aggregates.sum(salaries)}"></span>    <!-- 800 -->

<h4><code>#aggregates.avg(salaries)</code></h4>
<span th:text="${#aggregates.avg(salaries)}"></span>    <!-- environ 266,67 -->
```

On peut aussi agréger une propriété d'une liste d'objets, grâce à la **projection** SpringEL `.[...]` :

```
<p th:text="${#aggregates.sum(items.[price * quantity])}">Total</p>
```

#ids

Cet objet génère des identifiants uniques pour des éléments HTML répétés dans une boucle, afin de relier un `<label>` à son `<input>` sans collision :

```
<div th:each="color : ${colors}">
    <input type="checkbox" th:id="${#ids.seq('color')}" />
```

```
<label th:for="${#ids.prev('color')}}" th:text="${color}">Couleur</label>
</div>
```

`#ids.seq('color')` produit `color1`, `color2`..., et `#ids.prev('color')` rappelle le dernier identifiant généré.

Résumé

- Les objets prédéfinis se référencent par `#nom` dans les expressions : objets de base (`#ctx`, `#locale`) et objets utilitaires (`#strings`, `#numbers`, `#lists`, `#temporals`, ...).
- Depuis Thymeleaf 3.1, l'accès direct à l'API Servlet depuis un template est supprimé ; le contrôleur prépare les données de la vue.
- `#temporals` est l'objet à privilégier pour les dates modernes ; `#dates` et `#calendars` restent pour l'existant.
- Les objets utilitaires sont de simples aides : pour une logique complexe, préparez les données dans le contrôleur ou le service.

Exercices proposés

1. Afficher la langue et le pays de la requête, puis changer la langue du navigateur et observer le résultat.
2. Un ancien template contient `<span th:text="${#request.requestURI}">`. Le migrer vers Thymeleaf 3.1 avec un `@ControllerAdvice`.
3. Afficher une date `LocalDate` au format `dd/MM/yyyy` avec `#temporals`, puis un montant au format français (12 345,99) avec `#numbers.formatDecimal`.
4. Afficher la somme `prix × quantité` des lignes d'une commande avec `#aggregates` et une projection.
5. Quelle est la différence entre `#strings.isEmpty(x)`, `th:if="${x}"` et `${x} ?` : 'défaut' pour `x = ""` ? (voir les documents sur les conditions et l'opérateur Elvis)